



3

Working with Date and Time

This chapter includes 20 problems covering different date-time topics. These problems are mainly focused on the `Calendar` API and on the JDK Date/Time API. About the latter, we will cover some less popular APIs such as `ChronoUnit`, `ChronoField`, `IsoFields`, `TemporalAdjusters`, and so on.

At the end of this chapter, you'll have a ton of tips and tricks in your tool belt that will be very useful for solving a wide range of real-world date-time problems.

Problems

Use the following problems to test your programming prowess on date and time. I strongly encourage you to give each problem a try before you turn to the solutions and download the example programs:

68. Defining a day period: Write an application that goes beyond AM/PM flags and split the day into four periods: *night*,

morning, afternoon, and evening. Depending on the given date-time and time zone generate one of these periods.

69. **Converting between Date and YearMonth:** Write an application that converts between `java.util.Date` and `java.time.YearMonth` and vice versa.
70. **Converting between int and YearMonth:** Let's consider that a `YearMonth` is given (for instance, 2023-02). Convert it to an integer representation (for instance, 24277) that can be converted back to `YearMonth`.
71. **Converting week/year to Date:** Let's consider that two integers are given representing a week and a year (for instance, week 10, year 2023). Write a program that converts 10-2023 to a `java.util.Date` via `Calendar` and to a `LocalDate` via the `WeekFields` API. Also, do vice versa: from a given `Date` / `LocalDate` extract the year and the week as integers.
72. **Checking for a leap year:** Let's consider that an integer is given representing a year. Write an application that checks if this year is a leap year. Provide at least three solutions.
73. **Calculating the quarter of a given date:** Let's consider that a `java.util.Date` is given. Write a program that returns the quarter containing this date as an integer (1, 2, 3, or 4) and as a string (Q1, Q2, Q3, or Q4).
74. **Getting the first and last day of a quarter:** Let's consider that a `java.util.Date` is given. Write a program that returns the first and last day of the quarter containing this

date. Represent the returned days as `Date` (implementation based on `Calendar`) and `LocalDate` (implementation based on the JDK 8 Date/Time API).

75. **Extracting the months from a given quarter:** Let's consider that a quarter is given (as an integer, a string (Q1, Q2, Q3, or Q4), or a `LocalDate`). Write a program that extracts the names of the months of this quarter.
76. **Computing pregnancy due date:** Write a pregnancy due date calculator.
77. **Implementing a stopwatch:** Write a program that implements a stopwatch via `System.nanoTime()` and via `Instant.now()`.
78. **Extracting the count of milliseconds since midnight:** Let's consider that a `LocalDateTime` is given. Write an application that counts the milliseconds passed from midnight to this `LocalDateTime`.
79. **Splitting a date-time range into equal intervals:** Let's assume that we have a date-time range given via two `LocalDateTime` instances, and an integer, `n`. Write an application that splits the given range into `n` equal intervals (`n` equal `LocalDateTime` instances).
80. **Explaining the difference between `Clock.systemUTC()` and `Clock.systemDefaultZone()`:** Explain via meaningful examples what is the difference between `systemUTC()` and `systemDefaultZone()`.

- 81. Displaying the names of the days of the week:** Display the names of the days of the week via the `java.text.DateFormatSymbols` API.
- 82. Getting the first and last day of the year:** Let's consider that an integer representing a year is given. Write a program that returns the first and last day of this year. Provide a solution based on the `Calendar` API and one based on the JDK 8 Date/Time API.
- 83. Getting the first and last day of the week:** Let's assume that we have an integer representing a number of weeks (for instance, 3 represents three consecutive weeks starting from the current date). Write a program that returns the first and last day of each week. Provide a solution based on the `Calendar` API and one based on the JDK 8 Date/Time API.
- 84. Calculating the middle of the month:** Provide an application containing a snippet based on the `Calendar` API, and one based on the JDK 8 Date/Time API for calculating the middle of the given month as a `Date`, respectively as a `LocalDate`.
- 85. Getting the number of quarters between two dates:** Let's consider that a date-time range is given via two `LocalDate` instances. Write a program that counts the number of quarters contained in this range.
- 86. Converting Calendar to LocalDateTime:** Write a program that converts the given `Calendar` into a `LocalDateTime`

(default time zone), respectively into a `ZonedDateTime` (for the Asia/Calcutta time zone).

87. Getting the number of weeks between two dates: Let's assume that we have a date-time range given as two `Date` instances or as two `LocalDateTime` instances. Write an application that returns the number of weeks contained in this range. For the `Date` range, write a solution based on the `Calendar` API, while for the `LocalDateTime` range, write a solution based on the JDK 8 Date/Time API.

The following sections describe solutions to the preceding problems. Remember that there usually isn't a single correct way to solve a particular problem. Also, remember that the explanations shown here include only the most interesting and important details needed to solve the problems. Download the example solutions to see additional details and to experiment with the programs at

<https://github.com/PacktPublishing/Java-Coding-Problems-Second-Edition/tree/main/Chapter03>.

68. Defining a day period

Let's imagine that we want to say hello to a friend from another country (in a different time zone) via a message such as *Good morning*, *Good afternoon*, and so on based on their local time. So, having access to AM/PM flags is not enough, because

we consider that a day (24 hours) can be represented by the following periods:

- 9:00 PM (or 21:00) – 5:59 AM = night
- 6:00 AM – 11:59 AM = morning
- 12:00 PM – 5:59 PM (or 17:59) = afternoon
- 6:00 PM (or 18:00) – 8:59 PM (or 20:59) = evening

Before JDK 16

First, we have to obtain the time corresponding to our friend's time zone. For this, we can start from our local time given as a `java.util.Date`, `java.time.LocalDateTime`, and so on. If we start from a `java.util.Date`, then we can obtain the time in our friend's time zone as follows:

```
LocalTime lt = date.toInstant().atZone(zoneId).toLocalTime();
```

Here, `date` is a new `Date()` and `zoneId` is `java.time.ZoneId`. Of course, we can pass the zone ID as a `String` and use the `ZoneId.of(String zoneId)` method to get the `ZoneId` instance.

If we prefer to start from `LocalTime.now()`, then we can obtain the time in our friend's time zone as follows:

```
LocalTime lt = LocalTime.now(zoneId);
```

Next, we can define the day periods as a bunch of `LocalTime` instances and add some conditions to determine the current period. The following code exemplifies this statement:

```
public static String toDayPeriod(Date date, ZoneId zoneId) {  
    LocalTime lt = date.toInstant().atZone(zoneId).toLocalTime();  
    LocalTime night = LocalTime.of(21, 0, 0);  
    LocalTime morning = LocalTime.of(6, 0, 0);  
    LocalTime afternoon = LocalTime.of(12, 0, 0);  
    LocalTime evening = LocalTime.of(18, 0, 0);  
    LocalTime almostMidnight = LocalTime.of(23, 59, 59);  
    LocalTime midnight = LocalTime.of(0, 0, 0);  
    if((lt.isAfter(night) && lt.isBefore(almostMidnight))  
        || lt.isAfter(midnight) && (lt.isBefore(morning))) {  
        return "night";  
    } else if(lt.isAfter(morning) && lt.isBefore(afternoon)) {  
        return "morning";  
    } else if(lt.isAfter(afternoon) && lt.isBefore(evening)) {  
        return "afternoon";  
    } else if(lt.isAfter(evening) && lt.isBefore(night)) {  
        return "evening";  
    }  
    return "day";  
}
```

Now, let's see how we can do this in JDK 16+.

JDK 16+

Starting with JDK 16+, we can go beyond AM/PM flags via the following strings: *in the morning*, *in the afternoon*, *in the evening*, and *at night*.

These friendly outputs are available via the new pattern, `B`. This pattern is available starting with JDK 16+ via `DateTimeFormatter` and `DateTimeFormatterBuilder` (you should be familiar with these APIs from *Chapter 1, Problem 18*, shown in *Figure 1.18*).

So, the following code uses the `DateTimeFormatter` to exemplify the usage of pattern `B`, representing a period of the day:

```
public static String toDayPeriod(Date date, ZoneId zoneId) {  
    ZonedDateTime zdt = date.toInstant().atZone(zoneId);  
    DateTimeFormatter formatter  
        = DateTimeFormatter.ofPattern("yyyy-MM-dd [B]");  
    return zdt.withZoneSameInstant(zoneId).format(formatter);  
}
```

Here is an output for Australia/Melbourne:

```
2023-Feb-04 at night
```

You can see more examples in the bundled code. Feel free to challenge yourself to adjust this code to reproduce the result from the first example.

69. Converting between Date and YearMonth

Converting a `java.util.Date` to JDK 8 `java.time.YearMonth` can be done based on `YearMonth.from(TemporalAccessor temporal)`. A `TemporalAccessor` is an interface (more precisely, a framework-level interface) that exposes read-only access to any temporal object including date, time, and offset (a combination of these is also allowed). So, if we convert the given `java.util.Date` to `java.time.LocalDate`, then the result of the conversion can be passed to `YearMonth.from()` as follows:

```
public static YearMonth toYearMonth(Date date) {  
    return YearMonth.from(date.toInstant()  
        .atZone(ZoneId.systemDefault())  
        .toLocalDate());  
}
```

Vice versa can be obtained via `Date.from(Instant instant)` as follows:

```
public static Date toDate(YearMonth ym) {  
    return Date.from(ym.atDay(1).atStartOfDay(  
        ZoneId.systemDefault()).toInstant());  
}
```

Well, that was easy, wasn't it?

70. Converting between int and YearMonth

Consider that we have `YearMonth.now()` and we want to convert it to an integer (for example, this can be useful for storing a year/month date in a database using a numeric field). Check out the solution:

```
public static int to(YearMonth u) {  
    return (int) u.getLong(ChronoField.PROLEPTIC_MONTH);  
}
```

The *proleptic-month* is a `java.time.temporal.TemporalField`, which basically represents a date-time field such as *month-of-year* (our case) or *minute-of-hour*. The proleptic-month starts from 0 and counts the months sequentially from year 0. So, `getLong()` returns the value of the specified field (here, the proleptic-month) from this year-month as a `long`. We can cast this `long` to `int` since the proleptic-month shouldn't go be-

yond the `int` domain (for instance, for 2023/2 the returned `int` is 24277).

Vice versa can be accomplished as follows:

```
public static YearMonth from(int t) {  
    return YearMonth.of(1970, 1)  
        .with(ChronoField.PROLEPTIC_MONTH, t);  
}
```

You can start from any year/month. The 1970/1 (known as the *epoch* and the starting point for `java.time.Instant`) choice was just an arbitrary choice.

71. Converting week/year to Date

Let's consider the year 2023, week 10. The corresponding date is Sun Mar 05 15:15:08 EET 2023 (of course, the time component is relative). Converting the year/week to `java.util.Date` can be done via the `Calendar` API as in the following self-explanatory snippet of code:

```
public static Date from(int year, int week) {  
    Calendar calendar = Calendar.getInstance();  
    calendar.set(Calendar.YEAR, year);  
    calendar.set(Calendar.WEEK_OF_YEAR, week);  
    calendar.set(Calendar.DAY_OF_WEEK, 1);  
}
```

```
return calendar.getTime();  
}
```

If you prefer to obtain a `LocalDate` instead of a `Date` then you can easily perform the corresponding conversion or you can rely on `java.time.temporal.WeekFields`. This API exposes several fields for working with *week-of-year*, *week-of-month*, and *day-of-week*. This being said, here is the previous solution written via `WeekFields` to return a `LocalDate`:

```
public static LocalDate from(int year, int week) {  
    WeekFields weekFields = WeekFields.of(Locale.getDefault());  
    return LocalDate.now()  
        .withYear(year)  
        .with(weekFields.weekOfYear(), week)  
        .with(weekFields.dayOfWeek(), 1);  
}
```

On the other hand, if we have a `java.util.Date` and we want to extract the year and the week from it, then we can use the `Calendar` API. Here, we extract the year:

```
public static int getYear(Date date) {  
    Calendar calendar = Calendar.getInstance();  
    calendar.setTime(date);
```



```
    return calendar.get(Calendar.YEAR);  
}
```

And here, we extract the week:

```
public static int getWeek(Date date) {  
    Calendar calendar = Calendar.getInstance();  
    calendar.setTime(date);  
    return calendar.get(Calendar.WEEK_OF_YEAR);  
}
```

Getting the year and the week from a `LocalDate` is easy thanks to `ChronoField.YEAR` and `ChronoField.ALIGNED_WEEK_OF_YEAR`:

```
public static int getYear(LocalDate date) {  
    return date.get(ChronoField.YEAR);  
}  
public static int getWeek(LocalDate date) {  
    return date.get(ChronoField.ALIGNED_WEEK_OF_YEAR);  
}
```

Of course, getting the week can be accomplished via `WeekFields` as well:

```
return date.get(WeekFields.of(
    Locale.getDefault()).weekOfYear());
```

Challenge yourself to obtain week/month and day/week from a `Date` / `LocalDate`.

72. Checking for a leap year

This problem becomes easy as long as we know what a leap year is. In a nutshell, a leap year is any year divisible by 4 (so, `year % 4 == 0`) that it is not a century (for instance, 100, 200, ..., n00). However, if the year represents a century that is divisible by 400 (so, `year % 400 == 0`), then it is a leap year. In this context, our code is just a simple chain of `if` statements as follows:

```
public static boolean isLeapYear(int year) {
    if (year % 4 != 0) {
        return false;
    } else if (year % 400 == 0) {
        return true;
    } else if (year % 100 == 0) {
        return false;
    }
    return true;
}
```

But, this code can be condensed using the `GregorianCalendar` as well:

```
public static boolean isLeapYear(int year) {  
    return new GregorianCalendar(year, 1, 1).isLeapYear(year);  
}
```

Or, starting with JDK 8, we can rely on the `java.time.Year` API as follows:

```
public static boolean isLeapYear(int year) {  
    return Year.of(year).isLeap();  
}
```

In the bundled code, you can see more approaches.

73. Calculating the quarter of a given date

A year has 4 quarters (commonly denoted as Q1, Q2, Q3, and Q4) and each quarter has 3 months. If we consider that January is 0, February is 1, ..., and December is 11, then we can observe that $\text{January}/3 = 0$, $\text{February}/3 = 0$, $\text{March}/3 = 0$, and 0 can represent Q1. Next, $3/3 = 1$, $4/3 = 1$, $5/3 = 1$, so 1 can represent Q2. Based on the same logic, $6/3 = 2$, $7/3 = 2$, $8/3 = 2$, so 2

can represent Q3. Finally, $9/3 = 3$, $10/3 = 3$, $11/3 = 3$, so 3 represents Q4.

Based on this statement and the `Calendar` API, we can obtain the following code:

```
public static String quarter(Date date) {  
    String[] quarters = {"Q1", "Q2", "Q3", "Q4"};  
    Calendar calendar = Calendar.getInstance();  
    calendar.setTime(date);  
    int quarter = calendar.get(Calendar.MONTH) / 3;  
    return quarters[quarter];  
}
```

But, starting with JDK 8, we can rely on `java.time.temporal.IsoFields`. This class contains fields (and units) that follow the calendar system based on the ISO-8601 standard. Among these artifacts, we have the *week-based-year* and the one that we are interested in, *quarter-of-year*. This time, let's return the quarter as an integer:

```
public static int quarter(Date date) {  
    LocalDate localDate = date.toInstant()  
        .atZone(ZoneId.systemDefault()).toLocalDate();  
    return localDate.get(IsoFields.QUARTER_OF_YEAR);  
}
```

In the bundled code, you can see more examples including one that uses `DateTimeFormatter.ofPattern("QQQ")`.

74. Getting the first and last day of a quarter

Let's assume that we represent the first and last day of a quarter via this simple class:

```
public final class Quarter {  
    private final Date firstDay;  
    private final Date lastDay;  
    ...  
}
```

Next, we have a `java.util.Date` and we want the first and the last day of the quarter containing this date. For this, we can use JDK 8's `IsoFields.DAY_OF_QUARTER` (we introduced `IsoFields` in the previous problem). But, before we can use `IsoFields`, we have to convert the given `java.util.Date` to a `LocalDate` as follows:

```
LocalDate localDate = date.toInstant()  
    .atZone(ZoneId.systemDefault()).toLocalDate();
```

Once we have the given `Date` as a `LocalDate`, we can easily extract the first day of the quarter via `IsoFields.DAY_OF_QUARTER`. Next, we add 2 months to this day to move into the last month of the quarter (a quarter has 3 months, so a year has 4 quarters) and we rely on `java.time.temporal.TemporalAdjusters`, more precisely on `lastDayOfMonth()` to obtain the last day of the quarter. Finally, we convert the two obtained `LocalDate` instances to `Date` instances. Here is the complete code:

```
public static Quarter quarterDays(Date date) {
    LocalDate localDate = date.toInstant()
        .atZone(ZoneId.systemDefault()).toLocalDate();
    LocalDate firstDay
        = localDate.with(IsoFields.DAY_OF_QUARTER, 1L);
    LocalDate lastDay = firstDay.plusMonths(2)
        .with(TemporalAdjusters.lastDayOfMonth());
    return new Quarter(
        Date.from(firstDay.atStartOfDay(
            ZoneId.systemDefault()).toInstant()),
        Date.from(lastDay.atStartOfDay(
            ZoneId.systemDefault()).toInstant())
    );
}
```

Of course, these conversions are not needed if you work directly with `LocalDate`. But, this way, you have a chance to

learn more.

In the bundled code, you can find more examples, including one that relies entirely on the `Calendar` API.

75. Extracting the months from a given quarter

This problem becomes quite easy to solve if we are familiar with JDK 8's `java.time.Month`. Via this API, we can find the first month (0 for January, 1 for February, ...) of a quarter containing the given `LocalDate` as

```
Month.from(LocalDate).firstMonthOfQuarter().getValue()  
)
```

Once we have the first month, it is easy to obtain the other two as follows:

```
public static List<String> quarterMonths(LocalDate ld) {  
    List<String> qmonths = new ArrayList<>();  
    int qmonth = Month.from(ld)  
        .firstMonthOfQuarter().getValue();  
    qmonths.add(Month.of(qmonth).name());  
    qmonths.add(Month.of(++qmonth).name());  
    qmonths.add(Month.of(++qmonth).name());  
    return qmonths;  
}
```

How about passing the quarter itself as an argument? This can be done as a number (1, 2, 3, or 4) or as a string (Q1, Q2, Q3, or Q4). If the given `quarter` is a number, then the first month of the quarter can be obtained as `quarter * 3 - 2`, where the `quarter` is 1, 2, 3, or 4. This time, let's express the code in a functional style:

```
int qmonth = quarter * 3 - 2;
List<String> qmonths = IntStream.of(
    qmonth, ++qmonth, ++qmonth)
    .mapToObj(Month::of)
    .map(Month::name)
    .collect(Collectors.toList());
```

Of course, if you find it more concise, then you can use `IntStream.range(qmonth, qmonth+2)` instead of `IntStream.of()`. In the bundled, code you can find more examples.

76. Computing pregnancy due date

Let's start with these two constants:

```
public static final int PREGNANCY_WEEKS = 40;
public static final int PREGNANCY_DAYS = PREGNANCY_WEEKS * 7;
```


Let's consider the first day as a `LocalDate` and we want to write a calculator that prints the pregnancy due date, the number of remaining days, the number of passed days, and the current week.

Basically, the pregnancy due date is obtained by adding the `PREGNANCY_DAYS` to the given first day. Further, the number of remaining days is the difference between today and the given first day, while the number of passed days is `PREGNANCY_DAYS` minus the number of remaining days. Finally, the current week is obtained as the number of passed days divided by 7 (since a week has 7 days). Based on these statements, the code speaks for itself:

```
public static void pregnancyCalculator(LocalDate firstDay) {  
    firstDay = firstDay.plusDays(PREGNANCY_DAYS);  
    System.out.println("Due date: " + firstDay);  
    LocalDate today = LocalDate.now();  
    long betweenDays =  
        Math.abs(ChronoUnit.DAYS.between(firstDay, today));  
    long diffDays = PREGNANCY_DAYS - betweenDays;  
    long weekNr = diffDays / 7;  
    long weekPart = diffDays % 7;  
    String week = weekNr + " | " + weekPart;  
    System.out.println("Days remaining: " + betweenDays);  
    System.out.println("Days in: " + diffDays);  
}
```

```
System.out.println("Week: " + week);  
}
```

See if you can think of a way to use this to calculate another important date.

77. Implementing a stopwatch

A classical implementation for a stopwatch relies on `System.nanoTime()`, `System.currentTimeMillis()`, or on `Instant.now()`. In all cases, we have to provide support for starting and stopping the stopwatch, and some helpers to obtain the measured time in different time units.

While the solutions based on `Instant.now()` and `currentTimeMillis()` are available in the bundled code, here we'll show the one based on `System.nanoTime()`:

```
public final class NanoStopwatch {  
    private long startTime;  
    private long stopTime;  
    private boolean running;  
    public void start() {  
        this.startTime = System.nanoTime();  
        this.running = true;  
    }  
    public void stop() {  
        this.stopTime = System.nanoTime();  
    }  
}
```

```
        this.running = false;
    }
    //elapsed time in nanoseconds
    public long getElapsedTime() {
        if (running) {
            return System.nanoTime() - startTime;
        } else {
            return stopTime - startTime;
        }
    }
}
```

If you need to return the measured time in milliseconds or seconds as well, then simply add the following two helpers:

```
//elapsed time in milliseconds
public long elapsedTimeToMillis(long nanotime) {
    return TimeUnit.MILLISECONDS.convert(
        nanotime, TimeUnit.NANOSECONDS);
}
//elapsed time in seconds
public long elapsedTimeToSeconds(long nanotime) {
    return TimeUnit.SECONDS.convert(
        nanotime, TimeUnit.NANOSECONDS);
}
```

This approach is based on `System.nanoTime()` for measuring the elapsed time with high accuracy. This approach returns a

high-resolution time in nanoseconds that doesn't rely on a system clock or any other wall clock (as `Instant.now()` or `System.currentTimeMillis()` does), so it is not exposed to common issues of wall clocks such as leap second, time uniformity, synchronicity issues, and so on.

Whenever you need a professional tool for measuring elapsed time, rely on Micrometer (<https://micrometer.io/>), JMH (<https://openjdk.org/projects/code-tools/jmh/>), Gatling (<https://gatling.io/open-source/>), and so on.

78. Extracting the count of milliseconds since midnight

So, we have a date-time (let's say a `LocalDateTime` or `LocalTime`) and we want to know how many milliseconds have passed from midnight to this date-time. Let's consider that the given date-time is right now:

```
LocalDateTime now = LocalDateTime.now();
```

Midnight is relative to `now`, so we can find the difference as follows:

```
LocalDateTime midnight = LocalDateTime.of(now.getYear(),
```

```
now.getMonth(), now.getDayOfMonth(), 0, 0, 0);
```

Finally, compute the difference in milliseconds between midnight and now. This can be accomplished in several ways, but probably the most concise solution relies on

`java.time.temporal.ChronoUnit`. This API exposes a set of units useful to manipulate a date, time, or date-time including milliseconds:

```
System.out.println("Millis: "  
    + ChronoUnit.MILLIS.between(midnight, now));
```

In the bundled code, you can see more examples of `ChronoUnit`.

79. Splitting a date-time range into equal intervals

Let's consider a date-time range (bounded by a start date and an end date represented by two `LocalDateTime` instances) and an integer `n`. In order to split the given range into `n` equal intervals, we start by defining a `java.time.Duration` as follows:

```
Duration range = Duration.between(start, end);
```

Having this date-time range, we can rely on `dividedBy()` to obtain a copy of it divided by the specified `n`:

```
Duration interval = range.dividedBy(n - 1);
```

Finally, we can begin from the start date (the left head of the range) and repeatedly increment it with the `interval` value until we reach the end date (the right head of the range). After each step, we store the new date in a list that will be returned at the end. Here is the complete code:

```
public static List<LocalDateTime> splitInEqualIntervals(
    LocalDateTime start, LocalDateTime end, int n) {
    Duration range = Duration.between(start, end);
    Duration interval = range.dividedBy(n - 1);
    List<LocalDateTime> listOfDates = new ArrayList<>();
    LocalDateTime timeline = start;
    for (int i = 0; i < n - 1; i++) {
        listOfDates.add(timeline);
        timeline = timeline.plus(interval);
    }
    listOfDates.add(end);
    return listOfDates;
}
```

The resulting `listOfDates` will contain `n` dates at equal intervals.

80. Explaining the difference between `Clock.systemUTC()` and `Clock.systemDefaultZone()`

Let's start with the following three lines of code:

```
System.out.println(Clock.systemDefaultZone());  
System.out.println(system(ZoneId.systemDefault()));  
System.out.println(Clock.systemUTC());
```

The output reveals that the first two lines are similar. Both of them display the default time zone (in my case, Europe/Bucharest):

```
SystemClock[Europe/Bucharest]  
SystemClock[Europe/Bucharest]
```

The third line is different. Here, we see `Z` time zone, which is specific to the UTC time zone and indicates the presence of a zone offset:

```
SystemClock[Z]
```

On the other hand, creating an `Instant` reveals that `Clock.systemUTC()` and `Clock.systemDefaultZone()` produce the same result:

```
System.out.println(Clock.systemDefaultZone().instant());  
System.out.println(system(ZoneId.systemDefault()).instant());  
System.out.println(Clock.systemUTC().instant());
```

The instant time is the same in all three cases:

```
2023-02-07T05:26:17.374159500Z  
2023-02-07T05:26:17.384811300Z  
2023-02-07T05:26:17.384811300Z
```

But, the difference occurs when we try to create a date, time, or date-time from these two clocks. For instance, let's create a `LocalDateTime` from `Clock.systemUTC()`:

```
// 2023-02-07T05:26:17.384811300  
System.out.println(LocalDateTime.now(Clock.systemUTC()));
```

And, a `LocalDateTime` from `Clock.systemDefaultZone()`:

```
// 2023-02-07T07:26:17.384811300  
System.out.println(LocalDateTime.now(
```



```
Clock.systemDefaultZone()));
```

My time (default time zone, Europe/Bucharest) is 07:26:17. But, the time via `Clock.systemUTC()` is 05:26:17. This is because Europe/Bucharest is at an offset of UTC-2, so `systemUTC()` produces the date-time in the UTC time zone, while `systemDefaultZone()` produces the date-time in the current default time zone. However, both of them produce the same `Instant`.

81. Displaying the names of the days of the week

One of the hidden gems in Java is

`java.text.DateFormatSymbols`. This class is a wrapper for date-time formatting data such as the names of the days of the week, and the names of the months. All these names are localizable.

Typically, you will use `DateFormatSymbols` via a `DateFormat` such as `SimpleDateFormat`, but in order to solve this problem, we can use it directly as in the following code:

```
String[] weekdays = new DateFormatSymbols().getWeekdays();
IntStream.range(1, weekdays.length)
    .mapToObj(t -> String.format("Day: %d -> %s",
```

```
t, weekdays[t]))  
.forEach(System.out::println);
```

This code will output the weekdays' names as follows:

```
Day: 1 -> Sunday  
...  
Day: 7 -> Saturday
```

Challenge yourself to come up with another solution.

82. Getting the first and last day of the year

Getting the first and last day of the given year (as a numeric value) can be done via `LocalDate` and the handy `TemporalAdjusters`, `firstDayOfYear()`, and `lastDayOfYear()`. First, we create a `LocalDate` from the given year. Next, we use this `LocalDate` with `firstDayOfYear()` / `lastDayOfYear()` as in the following code:

```
public static String fetchFirstDayOfYear(int year, boolean name) {  
    LocalDate ld = LocalDate.ofYearDay(year, 1);  
    LocalDate firstDay = ld.with(firstDayOfYear());  
    if (!name) {  
        return firstDay.toString();  
    }  
}
```

```
}  
    return DateTimeFormatter.ofPattern("EEEE").format(firstDay);  
}
```

And, for the last day, the code is almost similar:

```
public static String fetchLastDayOfYear(int year, boolean name) {  
    LocalDate ld = LocalDate.ofYearDay(year, 31);  
    LocalDate lastDay = ld.with(lastDayOfYear());  
    if (!name) {  
        return lastDay.toString();  
    }  
    return DateTimeFormatter.ofPattern("EEEE").format(lastDay);  
}
```

If the flag argument (`name`) is `false`, then we return the first/last day via `LocalDate.toString()`, so we will get something like 2020-01-01 (the first day of 2020) and 2020-12-31 (the last day of 2020). If this flag argument is `true`, then we rely on the `EEEE` pattern to return only the name of the first/last day of the year as Wednesday (the first day of 2020) and Thursday (the last day of 2020).

In the bundle code, you can also find a solution to this problem via the `Calendar` API.

83. Getting the first and last day of the week

Let's assume given an integer (`nrOfWeeks`) representing the number of weeks that we want to extract the first and last day of each week starting from now. For instance, for the given `nrOfWeeks` = 3 and a local date such as 06/02/2023, we want this:

```
[  
  Mon 06/02/2023,  
  Sun 12/02/2023,  
  Mon 13/02/2023,  
  Sun 19/02/2023,  
  Mon 20/02/2023,  
  Sun 26/02/2023  
]
```

This is much easier than it might seem. We just need a loop from 0 to `nrOfWeeks` and two `TemporalAdjusters` to fit the first/last day of each week. More precisely, we need the `nextOrSame(DayOfWeek dayOfWeek)` and `previousOrSame(DayOfWeek dayOfWeek)` adjusters.

The `nextOrSame()` adjuster's role is to adjust the current date to the first occurrence of the given *day of week* after the date

being adjusted (this can be *next or same*). On the other hand, the `previousOrSame()` adjuster's role is to adjust the current date to the first occurrence of the given *day of week* before the date being adjusted (this can be *previous or same*). For instance, if today is [Tuesday 07/02/2023], then `previousOrSame(DayOfWeek.MONDAY)` will return [Monday 06/02/2023], and `nextOrSame(DayOfWeek.SUNDAY)` will return [Sunday 12/02/2023].

Based on these statements, we can solve our problem via the following code:

```
public static List<String> weekBoundaries(int nrOfWeeks) {  
    List<String> boundaries = new ArrayList<>();  
    LocalDate timeline = LocalDate.now();  
    DateTimeFormatter dtf = DateTimeFormatter  
        .ofPattern("EEE dd/MM/yyyy");  
    for (int i = 0; i < nrOfWeeks; i++) {  
        boundaries.add(dtf.format(timeline.with(  
            previousOrSame(DayOfWeek.MONDAY))));  
        boundaries.add(dtf.format(timeline.with(  
            nextOrSame(DayOfWeek.SUNDAY))));  
        timeline = timeline.plusDays(7);  
    }  
    return boundaries;  
}
```

In the bundled code, you can also see a solution based on the `Calendar` API.

84. Calculating the middle of the month

Let's imagine that we have a `LocalDate` and we want to calculate from it another `LocalDate` representing the middle of the month. This can be achieved in seconds if we know that the `LocalDate` API has a method named `lengthOfMonth()`, which returns an integer representing the length of the month in days. So, all we have to do is calculate `lengthOfMonth() / 2` as in the following code:

```
public static LocalDate middleOfTheMonth(LocalDate date) {  
    return LocalDate.of(date.getYear(), date.getMonth(),  
        date.lengthOfMonth() / 2);  
}
```

In the bundled code, you can see a solution based on the `Calendar` API.

85. Getting the number of quarters between two dates

This is just another problem that requires us to have a deep grasp of the Java Date/Time API. This time, we talk about `java.time.temporal.IsoFields`, which was introduced in *Problem 73*. One of the ISO fields is `QUARTER_YEARS`, which is a temporal unit representing the concept of a *quarter-year*. So, having two `LocalDate` instances, we can write this:

```
public static long nrOfQuarters(  
    LocalDate startDate, LocalDate endDate) {  
    return IsoFields.QUARTER_YEARS.between(startDate, endDate);  
}
```

Feel free to challenge yourself to provide a solution for `java.util.Date` / `Calendar`.

86. Converting Calendar to LocalDateTime

In *Problem 68*, you saw that converting a `java.util.Date` (date) to a `LocalTime` can be done as follows:

```
LocalTime lt = date.toInstant().atZone(zoneId).toLocalTime();
```

In the same manner, we can convert a `java.util.Date` to a `LocalDateTime` (here, `zoneId` was replaced with

```
ZoneId.systemDefault() )):
```

```
LocalDateTime ldt = date.toInstant().atZone(
    ZoneId.systemDefault()).toLocalDateTime();
```

We also know that we can obtain a `java.util.Date` from a `Calendar` via the `getTime()` method. So, by gluing the pieces of the puzzle together, we obtain the following code:

```
public static LocalDateTime
    toLocalDateTime(Calendar calendar) {
    Date date = calendar.getTime();
    return date.toInstant().atZone(
        ZoneId.systemDefault()).toLocalDateTime();
}
```

The same result but following a shorter path can be obtained like this:

```
return LocalDateTime.ofInstant(Instant.ofEpochMilli(
    calendar.getTimeInMillis()), ZoneId.systemDefault());
```

Or, even shorter, like this:

```
return LocalDateTime.ofInstant(
```



```
calendar.toInstant(), ZoneId.systemDefault());
```

But, this code assumes that the time zone of the given `Calendar` is the default time zone. If the calendar has a different time zone (for instance, Asia/Calcutta), then we might expect back a `ZonedDateTime` instead of a `LocalDateTime`. This means that we should adjust the previous code accordingly:

```
public static ZonedDateTime  
    toZonedDateTime(Calendar calendar) {  
    Date date = calendar.getTime();  
    return date.toInstant().atZone(  
        calendar.getTimeZone().toZoneId());  
}
```

Again, some shorter versions are available, but we've not shown these as they are less expressive:

```
return ZonedDateTime.ofInstant(  
    Instant.ofEpochMilli(calendar.getTimeInMillis()),  
    calendar.getTimeZone().toZoneId());  
return ZonedDateTime.ofInstant(calendar.toInstant(),  
    calendar.getTimeZone().toZoneId());
```

Done!

87. Getting the number of weeks between two dates

If the given two dates are instances of `LocalDate ( Time )`, then we can rely on `java.time.temporal.ChronoUnit`. This API exposes a set of units useful to manipulate a date, time, or date-time and we have used it before in *Problem 78*. This time, let's use it again to compute the number of weeks between two dates:

```
public static long nrOfWeeks(  
    LocalDateTime startLdt, LocalDateTime endLdt) {  
    return Math.abs(ChronoUnit.WEEKS.between(  
        startLdt, endLdt));  
}
```

On the other hand, if the given dates are `java.util.Date`, then you can choose to convert them to `LocalDateTime` and use the previous code or to rely on the `Calendar` API. Using the `Calendar` API is about looping from the start date to the end date while incrementing the calendar date week by week:

```
public static long nrOfWeeks(Date startDate, Date endDate) {  
    Calendar calendar = Calendar.getInstance();  
    calendar.setTime(startDate);  
    int weeks = 0;  
    while (calendar.getTime().getTime() < endDate.getTime()) {  
        calendar.add(Calendar.WEEK, 1);  
        weeks++;  
    }  
    return weeks;  
}
```

```
while (calendar.getTime().before(endDate)) {  
    calendar.add(Calendar.WEEK_OF_YEAR, 1);  
    weeks++;  
}  
return weeks;  
}
```

When the calendar date is after the end date, we have the number of weeks.

Summary

Mission accomplished! I hope you enjoyed this short chapter filled to the brim with tips and tricks about manipulating date-time in real-world applications. I strongly encourage you to also read the homologous chapter from *Java Coding Problems, First Edition*, which contains another 20 problems covering other date-time topics.

Join our community on Discord

Join our community's Discord space for discussions with the author and other readers:

<https://discord.gg/8mgytp5DGQ>

